Cephadm Agent Status Report

Oct 31, 2026 · @Kyle

The agent rework works end to end. agentd takes agent traffic out of ceph-mgr. A Rust ceph-agent applies mgr-signed intents through Quadlet and systemd instead of SSH deploys. Both run a real Ceph cluster on three VMs, and in simulation they carry 10,000 hosts and 140,000 daemons. Today's work bounded agentd's memory and paced agent upgrades: in the simulator, 5,000 agents restarting at once overwhelmed SWIM.

Where things stand

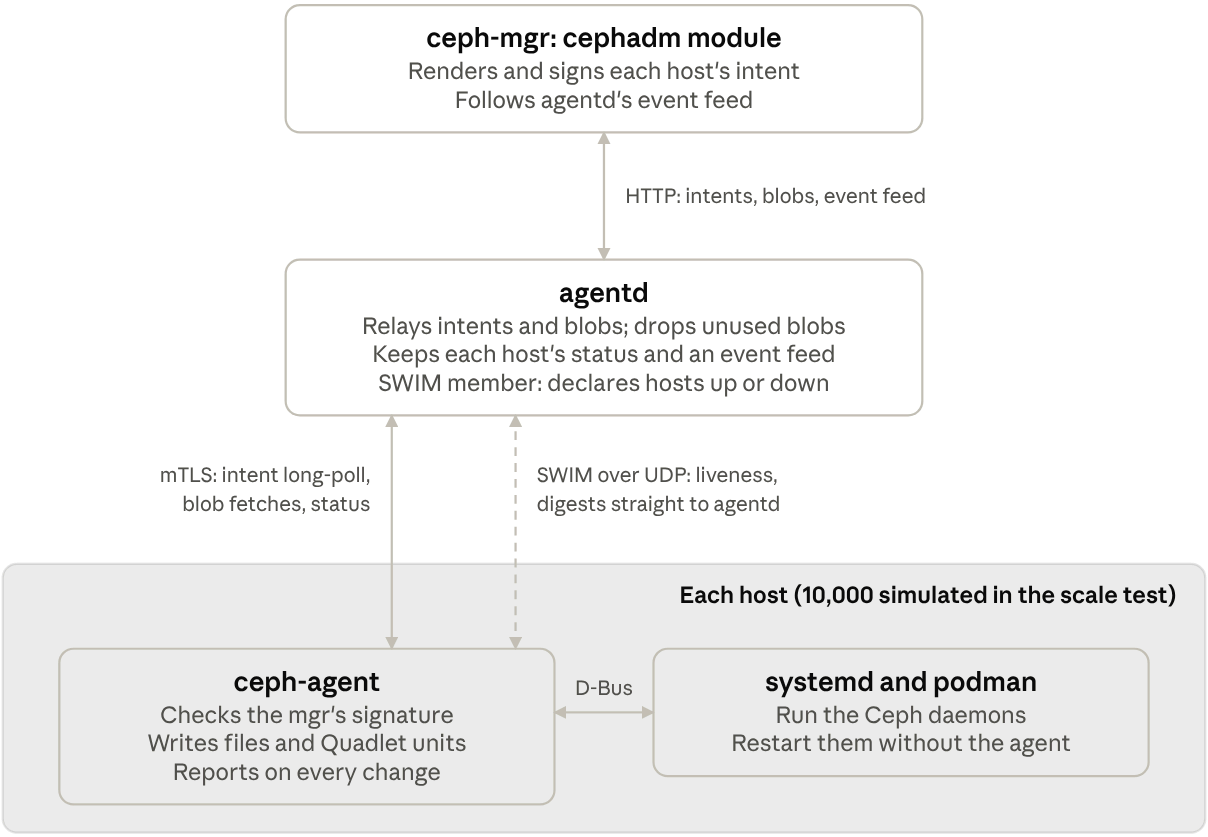
Everything below has been built and measured except cephadm's own side, which is designed but not written. The two newest pieces, blob cleanup and paced agent upgrades, were built and measured today.

Workstream	State	Result
Stock cephadm agent endpoint (baseline)	Measured	Keeps up to about 1,000 hosts at a 20 s refresh; 0% of reports delivered at 1,500
agentd, cephadm's v1 protocol	Measured	4,000 hosts at p99 3.2 ms and 0.2 cores
v2 protocol: one TLS connection, versioned deltas	Measured	50,000 hosts on 0.13 cores (v1: 1.67); 96 bytes per agent per period (v1: 50,360)
SWIM liveness (foca)	Measured	4,000 members: a dead host found in 16.8 s p50 (60 s today), no false downs
ceph-agent: signed intents applied through Quadlet and systemd	Prototype works	Runs a Tentacle 20.2.4 cluster on 3 VMs: 3 mons, 2 mgrs, 6 OSDs, HEALTH_OK
agentd orchestrator mgr module	Prototype works	`ceph orch ps`, `host ls`, `daemon restart` and AGENTD_* health checks on the VMs

Workstream	State	Result
Scale test of ceph-agent and agentd	Measured	10,000 hosts, 140,000 daemons: a ceph.conf change applied on every host in 2.55 s
Blob cleanup in agentd	Measured	10,000 hosts: an image change's 130,000 old blobs dropped in a 75 ms sweep; memory after deploy 2.43 GB → 1.34 GB
Paced agent upgrades	Measured	5,000 hosts at 20/s in 4.4 minutes, no false downs; all at once melted SWIM down in the simulator. A failed canary halted the VM rollout
cephadm consuming agentd	Design only	docs/cephadm-integration.md; its first step needs no agent change

How it fits together

The mgr signs intents, agentd relays them, each host applies them



components and the links between them

Agents also probe each other over SWIM, so a dead host is noticed by its peers rather than by missed heartbeats. The daemons are systemd units, so they keep running and restarting while the agent is down or being upgraded.

Baseline: the stock endpoint

The cephadm mgr module's agent endpoint stops keeping up at about 1,000 hosts with a 20 s refresh, and delivers no reports at 1,500 (ceph main 9e49f5cb896). 62% of the mgr's GIL time went to `get_error_daemons()`, which rescans every daemon three times per report.

- A small patch to the module (A+) raised capacity 4–5×. It moves the ceiling but does not remove it.

- There is no admission control: past capacity, goodput collapses and Python's garbage collector spirals.
- TLS handshakes are serialized on cheroot's accept thread.
- `orch ps --hostname` copies every daemon through a JSON round trip.
- The host inventory is a single config key, which fails with EFBIG past about 690 hosts.
- An RSA-4096 key is generated per mgr-to-agent message, and the endpoint port moves when a probe socket lacks SO_REUSEADDR.

The full benchmark is in the endpoint benchmark report.

agentd and the protocol

agentd, a Rust endpoint outside the mgr with the same validation and persistence, carries 4,000 hosts at p99 3.2 ms on 0.2 cores. Its peak was about 9,900 reports/s; the stock endpoint managed 17/s.

- **v2 protocol.** Each agent keeps one TLS connection, sends a full snapshot once, then versioned deltas and heartbeats. At 50,000 hosts agentd used 0.13 cores instead of 1.67, and each agent sent 96 bytes per period instead of 50,360. A change reached agentd in 1–3 ms. Heartbeats are still one per host per period.
- **SWIM liveness** (foca, LAN profile). With 4,000 members each sent 2.1 packets/s and agentd about 5. A dead host was declared down in 16.8 s p50 (cephadm today waits 60 s), with no false downs. A signal reached all 4,000 hosts in 0.44 s p50, 1.15 s max.
- **Encoding.** Parsing JSON is 1–3% of the mgr's CPU per report, so protobuf would save little. zlib at level 1 shrinks a report 8.7×; dropping field names saves about 40%.

ceph-agent on VMs

ceph-agent replaces SSH deploys, and it runs a real Ceph Tentacle 20.2.4 cluster on three Fedora 44 VMs (3 mons, 2 mgrs, 6 OSDs, HEALTH_OK). The mgr publishes a signed intent per host. The agent applies it locally and reports what systemd says.

- **Intents.** Quadlet `.container` files and config files by SHA-256, desired unit states, restart keys and removals, signed with Ed25519. The agent long-polls agentd, checks the signature, fetches the blobs, writes files atomically, reloads systemd and starts or restarts units over D-Bus.

- **Status.** The agent subscribes to systemd's `PropertiesChanged` on its units and posts on every material change. agentd keeps an event feed of unit transitions, applied generations and liveness.
- **Joining.** A one-time token, a CSR the mgr signs, and an X25519 key that secrets such as keyrings are sealed to. agentd only relays these.
- **Jobs** run once per id in a container (ceph-volume prepare, mon mkfs); units can wait for them.
- **Self-upgrade.** The intent names an agent binary. A binary that can't start is rolled back by systemd; one that can't reach agentd within 2 minutes rolls itself back.
- **mgr module.** An `agentd` orchestrator backend answers `ceph orch ps`, `host ls` and `daemon restart`, and raises AGENTD_* health checks.

On the VMs	Time
First deploy of a host, image pulls included	2.5–3.6 s
Config-only change, push to applied	55 ms
Daemon restart, push to applied	190 ms
`podman kill` of a daemon seen by the agent	77 ms
`ceph orch daemon restart osd.4` to a new OSD process	2.0 s
Frozen agent to `AGENTD_HOST_DOWN` / cleared after thaw	15.4 s / 1.6 s
Agent self-upgrade, push to committed	3.1–4.1 s

Scale test to 10,000 hosts

One agentd on 2 cores pushed a ceph.conf change to 10,000 hosts (140,000 daemons), and every host had applied it 2.55 s later. Each simulated host runs the real agent code over an in-memory systemd, from its own loopback address. Each has 12 OSDs, crash and node-exporter, and 13 keyrings sealed to it. The mgr side is Python, as cephadm is.

	1,000	2,500	5,000	10,000
Daemons	14,000	35,000	70,000	140,000

	1,000	2,500	5,000	10,000
Join storm: all hosts joined	2.2 s	3.3 s	6.9 s	9.3 s
ceph.conf change: all hosts applied	0.21 s	0.73 s	1.85 s	2.55 s
Per host, intent stored to applied, p99	0.11 s	0.20 s	0.43 s	0.45–0.56 s
Frozen rack of 40 hosts declared down	13.2–17.1 s	14.9–19.5 s	16.4–20.4 s	not run
Other hosts falsely declared down	0	0	0	not run
agentd `kill -9`: all hosts back and reporting	3.4 s	5.4 s	4.9 s	3.0 s
agentd CPU, idle	0.002 cores	0.003 cores	0.008 cores	0.010 cores
agentd CPU for the whole change	0.4 s	0.8 s	2.5 s	3.4 s
agentd memory after deploy, before today's blob fix (see Blob cleanup)	0.3 GB	0.7 GB	1.5 GB	2.4 GB

- 10,000 hosts ran without SWIM members: one machine can't simulate more than about 5,000 of them.
- An idle simulated agent costs 0.008% of a core and sends 2.3 SWIM packets/s (107 B/s). A real idle agent on the VMs used 0.18% of a core and 9 MiB.
- First deploys took 6.4–66.8 s to apply everywhere. That is the simulator's CPU: each host's apply costs about 50 ms, and all of them share 12 cores.
- At 10,000 hosts, the mgr module's read path takes 0.7 s of CPU and 9 requests to follow a cluster-wide change (it took 1.8 s and 11,654 requests before batch reads).

Blob cleanup in agentd

agentd now drops blobs that no host's current intent names, 10 minutes after they stop being named. At 10,000 hosts, a Ceph image change leaves 130,000 old Quadlet files (121 MB). One sweep dropped them in 75 ms, and agentd gave 141 MB back to the system.

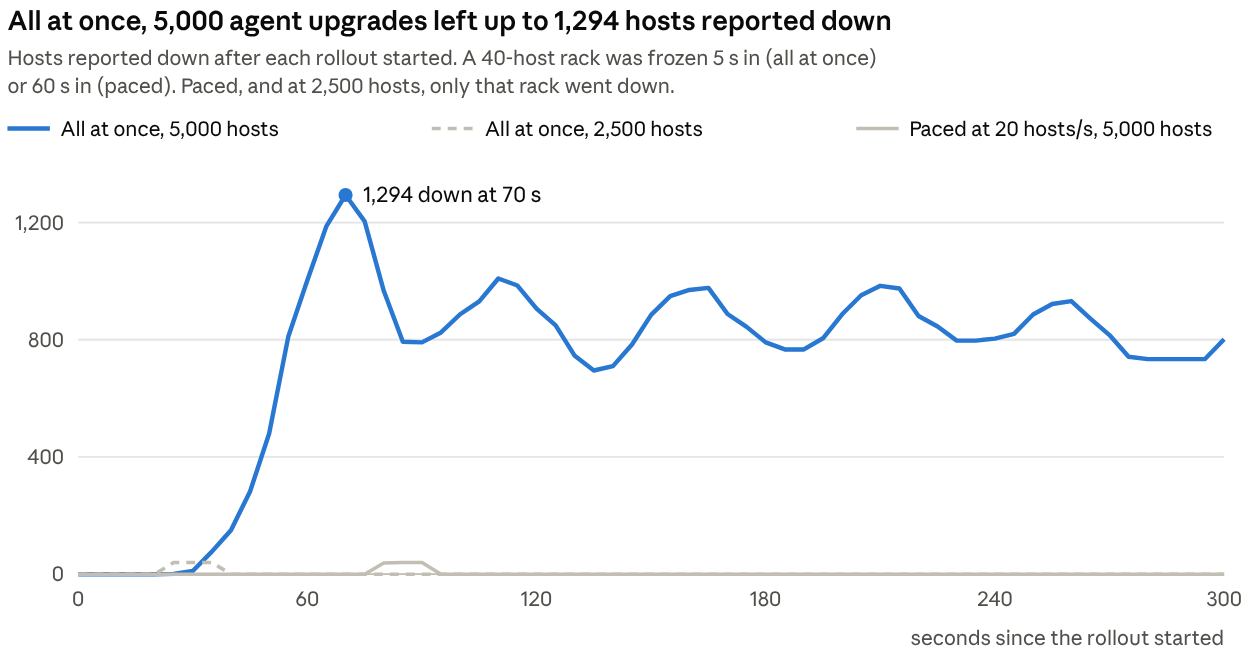
10,000 hosts, no SWIM	Before	Now
agentd memory after the first deploy	2.43 GB	1.34 GB
Memory per stored 1.1 KB blob	5.4 KB	1.3 KB
Old blobs after an image change	kept until agentd restarts	130,000 dropped after the grace period

- **Reference counts.** Each blob counts the current intents that name it. The count moves when a host's intent is replaced, under the same lock as the check that rejects intents naming missing blobs, so a sweep can't drop a blob between the two.
- **Grace period** (`--blob-grace-s`, default 600). The mgr uploads blobs before the intent that names them, and an agent still applying the previous generation may fetch one late.
- **Removed hosts.** `DELETE /v1/hosts/<host>` releases a removed host's intent, status and blobs.
- **The bigger leak.** A small blob kept the whole network read buffer it arrived in. Copying it when stored saved more memory than the cleanup itself.
- **Returning memory.** glibc keeps freed memory for reuse, so agentd calls `malloc_trim` after a sweep that freed more than 16 MB. That took 46 ms at 10,000 hosts.

Memory levels off. Over five image changes in a row at 10,000 hosts, agentd's memory after each sweep was 1,316, 1,390, 1,462, 1,493 and 1,508 MB, growing less each cycle. Before, every image change kept 130,000 more blobs for good, about 0.7 GB at the old cost per blob.

Paced agent upgrades

Agent upgrades need pacing, and the mgr now paces them. In the simulator, upgrading 5,000 agents at once left up to 1,294 hosts reported down, though only 40 had actually failed. At 20 hosts/s the same rollout took 4.4 minutes and reported only those 40.



scale test on one machine, 5,000 and 2,500 simulated hosts with SWIM; agentd's change feed sampled every 5 s

How the mgr paces a rollout (`intentctl push --canary --rate --max-in-flight --max-failures` , and the scale test's Python mgr):

1. Canaries get the new binary first. Any failure among them halts the rollout.
2. After a soak, the other hosts follow at a set rate, with a cap on hosts that have the binary but aren't running it yet.
3. A host is done when agentd's event feed shows an agent process on the new binary. It has failed if the agent reports a rollback or an upgrade error, or it times out. Too many failures halt the rollout.

5,000 hosts with SWIM	Paced, 20 hosts/s	All at once
All hosts on the new agent	261 s	6.5 s
Frozen rack declared down: first / p50 / max	16.1 / 17.7 / 21.2 s	25.6 / 53.6 / 112 s
Other hosts declared down	0	3,562
Simulator CPU, of 12 cores	6.6 cores	11.9 cores

- **The simulator's CPU is part of the story.** At 2,500 hosts, all at once did no harm: a 40 s burst of gossip, the rack down in 15–19 s, nothing else. At 5,000 the simulator ran out of CPU and SWIM melted down. Real hosts each have their own CPU, so where this starts

on real hardware is unknown and likely higher. The weakness is the same, though: a member that can't answer a probe in time gets suspected.

- **Any rollout fills the gossip channel.** At 20 hosts/s each member sent 16 packets/s, 7× its idle rate. Failure detection kept its speed because foca sends the freshest updates first.
- **On the VMs,** a canary whose new binary crashed was rolled back by systemd, and the rollout halted 2.6 s in, before node2 and node3. The real 0.2.5 agent then rolled out in 18.3 s (2.6 s per node) without restarting a daemon, and Ceph stayed HEALTH_OK.
- **Bandwidth isn't the limit:** 10,000 hosts fetched 80 GB of agent binaries from agentd in 11.9 s.

Bugs the tests found

The scale and VM tests found 12 bugs, all fixed; the last two were found today.

Bug	Effect	Fix
A stored blob kept the whole network read buffer it arrived in	Each 1 KB blob cost agentd 5.4 KB: about 1.3 GB at 10,000 hosts	agentd copies a blob when storing it: 1.3 KB per 1 KB blob
An agent-upgrade error was recorded after the change check, without waking the reporter	The error reached agentd late, and later idle passes counted as changes	The upgrade runs before the check, and its error is reported like any other
Rollback hooks stayed armed after an agent upgrade committed	Four admin restarts in 2 minutes silently downgraded the agent (VMs)	Hooks exist only while an upgrade is pending
First status after an agentd restart waited for the 30 s long-poll	Hosts reappeared 30 s late	The first poll after connecting returns at once: 2–3 s
A restarted agentd relearned SWIM members only as they probed it	19 of 2,500 hosts still not up after 300 s	Agents re-announce until agentd's status reply says it sees them; 4 MB UDP receive buffer

Bug	Effect	Fix
SWIM news for unknown members was filed under `member-<idx>`	1 of 1,000 hosts never shown up	Moved to the host when the mgr re-teaches the members
Every idle drift check counted as a change	Every host reported every minute	A pass that changes nothing doesn't report
Per-host digests were gossiped to every member	5,000 hosts never converged: 3,940 up after 15 minutes	Digests go straight to agentd as one datagram
The event ring held a fixed 100,000 events	A 10,000-host change overran it; events were lost	32 events per host, and a `first` field so readers know to resync
Listen backlog of 1,024	10,000-agent reconnects took 6.0 s	Backlog 8,192: 3.0 s
intentctl rewrote its state file per approved join	Joins were $O(N^2)$	One save per batch
The mgr module made one request per changed host	11,654 requests for one change at 10,000 hosts	One batch request: 9

Open items

The largest open item is cephadm itself: none of this is wired into it yet, and nothing has been posted upstream.

- **cephadm integration.** Service specs, placement and Ceph upgrades still go through SSH. The orchestrator backend here is a separate module. Its first step, reading agentd's change feed, needs no agent change (docs/cephadm-integration.md).
- **Paced rollouts in cephadm.** The controller exists in the test harness and in intentctl. In cephadm it would need state that survives a mgr failover.
- **More than 5,000 SWIM members** have not been measured: one machine can't simulate them. That needs several machines.
- **Real hardware.** All scale numbers come from one machine, and per-host apply times there are the simulator's CPU.
- **agentd is a single process with in-memory state.** A restart costs a 3–5 s reconnect and a republish by the mgr. There is no failover.

- **Quadlet needs podman 4.4 or later**, which rules out docker hosts and Ubuntu 22.04. Unit names change from `ceph-<fsid>@osd.17` to `ceph-<fsid>-osd.17`.
- **No security review** yet of join tokens, sealed secrets or the mTLS setup.

Sources

Everything lives in the `agentbench` tree (a local git repository, not yet committed), mirrored to the test machine, bravo (Ryzen AI Max+ 395, 16 cores).

- **Code:** `rust/agentd`, `rust/ceph-agent`, `rust/intentctl` (the mgr's part for tests), `rust/proto` (intents, join, sealing, SWIM), `vm/ceph/mgr-agentd` (orchestrator module).
- **Harness:** `bench/` for the endpoint benchmark; `bench/scale/` for the scale test (`run.sh`, `mgr.py`, `analyze.py`, `swimrate.py`); `vm/` for the three test VMs and the Ceph bootstrap.
- **Design and findings:** `docs/agent-executes-intent.md` (the agent design), `docs/cephadm-integration.md` (cephadm reading agentd), `docs/agent-scale-test.md` (scale test and fixes).
- **Results:** `results/scale/<run>/` on bravo, with copies in `results-bravo/`.
- **Versions:** baseline on ceph main 9e49f5cb896; the VM cluster runs Ceph Tentacle 20.2.4 on Fedora 44.
- **Earlier report:** endpoint benchmark.